



Sapienza Università di Roma
Corso di laurea in Ingegneria Informatica e Automatica

Tecnologie e Sistemi Web

a.a. 2025/2026

Parte 1

HTML

Lorenzo Marconi
Dipartimento di Ingegneria informatica, automatica e gestionale
Sapienza Università di Roma

Sommario

1. World Wide Web
2. Iper testi e HTML
3. HTML di base
4. Form
5. HTML5

1. World Wide Web

Il World Wide Web

- World Wide Web = sistema di accesso a Internet basato sul protocollo HTTP
 - insieme di protocolli e servizi (HTTP, FTP, ...)
 - insieme di tool per l'accesso (es. web browser)
- Basato sulla metafora dell'**ipertesto**
 - ⇒ linguaggio HTML
- Distinzione tra Internet e WWW
 - Internet = rete
 - WWW = sistema di accesso alla rete

Architettura del web

Architettura client-server:

- Web client (es. web browser)
 - inoltra richieste di **risorse** (documenti, file, ecc.) ad una macchina (web server)
- Web server
 - risponde alle richieste dei web client
- Sia il web server che il web client sono programmi in esecuzione su macchine connesse a Internet
- Web server e web client comunicano in base al protocollo HTTP

Architettura client-server

- Basata sul concetto di **richiesta** di servizio (client) e di **fornitura** di servizio (server)
- Enormemente diffuse in informatica, in particolare nelle applicazioni di rete
 - HTTP
 - FTP
 - DNS
 - PPP
 - Proxy
 -

Protocolli

- Protocollo = insieme di convenzioni (o regole) per lo scambio di informazioni
- protocolli di “basso” livello per Internet:
 - determinano le modalità di comunicazione tra i nodi della rete
 - TCP/IP
- protocolli di “alto” livello:
 - determinano il formato dei messaggi e le modalità di scambio dei messaggi
 - HTTP, FTP, SMTP, TELNET...

Il protocollo HTTP

HTTP = HyperText Transfer Protocol

- Client-server
 - ogni interazione è una richiesta (messaggio ASCII) seguita da una risposta (messaggio tipo MIME)
- 9 metodi:
 - GET
 - POST
 - PUT
 - DELETE
 - PATCH
 - HEAD
 - OPTIONS
 - TRACE
 - CONNECT

Metodi HTTP

- GET
 - Recupera una risorsa dal server
 - Non modifica i dati
 - I parametri sono visibili nell'URL
 - Deve essere idempotente (stessa richiesta = stesso effetto)
- POST
 - Invia dati al server
 - Crea nuove risorse o avvia elaborazioni
 - I dati sono nel body della richiesta
 - Non è idempotente
- PUT
 - Aggiorna o sostituisce completamente una risorsa
 - È idempotente
- DELETE
 - Elimina una risorsa
 - È idempotente

Metodi HTTP (cont'd)

- PATCH
 - Modifica parzialmente una risorsa
 - Non sempre idempotente (dipende dall'implementazione)
- HEAD
 - Come GET, ma restituisce solo gli header (senza body)
 - Utile per controllare se una risorsa esiste
- OPTIONS
 - Restituisce i metodi supportati da una risorsa
 - Spesso usato nelle richieste CORS
- TRACE
 - Esegue un loopback della richiesta per debug
- CONNECT
 - Stabilisce un tunnel verso il server
 - Usato principalmente per HTTPS tramite proxy

Connessione HTTP

- Client-server
- HTTP è *connectionless* = **non** si instaura una connessione prima di richiedere un servizio
- FTP:
 - richiesta connessione
 - instaurazione connessione
 - richiesta servizio 1
 - fornitura servizio 1
 - richiesta servizio 2 ...
 - chiusura connessione
- HTTP:
 - richiesta servizio
 - fornitura servizio

URL

- In HTTP ogni interazione è relativa ad una **URL (Uniform Resource Locator)**
- La URL è un nome che identifica univocamente ogni risorsa disponibile sul web
- Ogni URL specifica:
 - il protocollo da utilizzare per il trasferimento della URL
 - il dominio, cioè il nome (simbolico) del computer su cui si trova il server (web server o altro server) che gestisce la risorsa
 - il nome, all'interno del dominio, della risorsa che si vuole accedere

Domain Name System (DNS)

- Sistema per introdurre nomi logici (o simbolici) ai computer su Internet
- IP (Internet Protocol):
 - l'indirizzo del computer è una sequenza di 4 numeri da 0 a 255
 - es. 151.100.59.104
- DNS:
 - l'indirizzo del computer è una stringa di caratteri
 - es. `www.diag.uniroma1.it`
- Il DNS è basato sul concetto di **dominio**

Domini

Domini radice o di primo livello:

- COM, ORG, NET, EDU, MIL, GOV, INT
- biletterale per ogni nazione (es. IT)

Per ogni dominio di primo livello:

- domini di **secondo livello** (es. IBM.COM, VIRGILIO.IT)
- ogni dominio di primo livello gestisce in modo autonomo i propri domini secondari
- domini di terzo livello, quarto, ecc.

i nomi dei domini sono case-insensitive

Name Server

- Occorre tradurre il nome simbolico in indirizzo IP
- NAME SERVER (o Domain Name Server)
- Si deve ricorrere ad un name server ogniqualevolta si fa uso di un nome simbolico
- Es. ogni web browser che richiede un'URL, deve **prima** richiedere ad un name server l'indirizzo IP corrispondente al dominio nell'URL:
- Dominio → Name Server → indirizzo IP

URL: esempio

URL: `http://www.diag.uniroma1.it/index.php`

`www.diag.uniroma1.it`



NAME SERVER

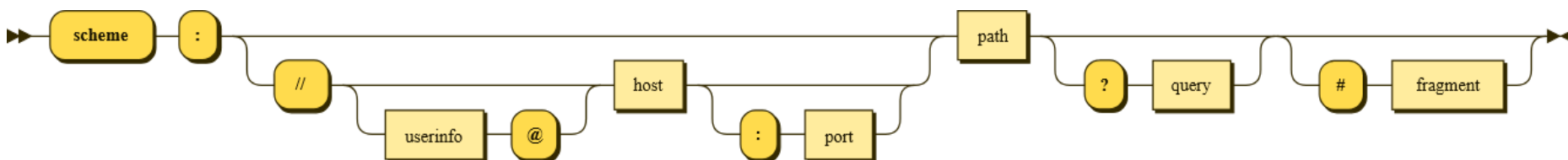


`151.100.59.104`

- `index.php` è il nome (completo di percorso) del file corrispondente alla URL

URI

- La nozione di **Uniform Resource Identifier (URI)** generalizza quella di URL
- Es. *mailto:marconi@diag.uniroma1.it*
- Lo schema è molto simile:



fonte: https://commons.wikimedia.org/wiki/File:URI_syntax_diagram.svg

Domini, siti web e pagine web

Distinzione tra domini, web server, siti web e pagine web:

- dominio = computer dove gira un web server (“identifica” il web server)
- **sito web** = insieme di URL gestite da un’unica entità e organizzate in modo da essere accedute secondo un ordine logico
 - più siti web possono essere gestiti dallo stesso web server
- **pagina web** = singolo documento HTML
- **home page** = pagina iniziale di un sito web

2. Iper testi e HTML

Iper testi

Iper testo = documento contenente:

- testo
- immagini
- audio
- video
- **collegamenti ipertestuali** = riferimenti ad altri documenti (o parti di documenti) ipertestuali

Collegamenti ipertestuali

- Sul World Wide Web, un collegamento ipertestuale ad un documento può essere specificato tramite la URL che corrisponde al documento
- Si possono usare le tecnologie informatiche per **accedere direttamente** ai documenti corrispondenti ai link mentre si legge un ipertesto
- (esempio: web browser)
- superamento dell'accesso "sequenziale" al testo

Linguaggio per ipertesti: HTML

- HTML = HyperText Markup Language
- Linguaggio standard per la specifica di documenti ipertestuali sul World Wide Web
- Linguaggio a marcatura, “figlio” di SGML (Standard Generalized Markup Language)
- Un documento HTML può contenere:
 - testo
 - link ipertestuali
 - immagini
 - link a risorse (URL) di ogni tipo

Breve storia di HTML (1)

- definito (insieme ad HTTP) da **Tim Berners-Lee** del CERN di Ginevra nel 1989
 - scopo: permettere lo scambio dei dati sperimentali tra i fisici, superando i problemi di *incompatibilità* tra sistemi, gestione dei *collegamenti* tra documenti e accesso *non centralizzato* alle informazioni
- 1991: pubblicazione del primo browser e del **primo sito web** (oggi accessibile all'URL <https://info.cern.ch/>)
- 1993: diffusione di Mosaic (web browser sviluppato da NCSA)
- 1995: fondazione del World Wide Web Consortium (W3C)
- 1999: standardizzazione di **HTML 4.0**
- 2000: standardizzazione di XHTML 1.0 (ridefinizione di HTML basata su XML)

Breve storia di HTML (2)

- 2004: viene fondato il **WHATWG**, Web Hypertext Application Technology Working Group
 - Apple, Mozilla Foundation, Opera Software, Google
- WHATWG nasce in contrapposizione con il futuro standard XHTML2 del W3C, che sarebbe stato un linguaggio non retrocompatibile con HTML 4
- ottobre 2014: standardizzazione di **HTML5**
- novembre 2016: standardizzazione di HTML 5.1
- dicembre 2017: standardizzazione di HTML 5.2
- maggio 2019: accordo tra W3C e WHATWG
 - Il «**living standard**» di HTML di WHATWG diventa il riferimento comune dei due consorzi

Sintassi di HTML

- Documento HTML: testo normalmente codificato in UTF-8 (storicamente in ASCII)
- contiene:
 - blocchi di testo
 - tag (marcature o “comandi”)
- tag = testo delimitato dai simboli “<” e “>”
- esempio:

`<nome-tag>`

Il concetto di tag

- TAG = “marcatura” (o marcatore)
- Un tag viene usato per **marcare** una parte di testo
- Per HTML i tag sono **case-insensitive** (es. `<img>` e `<IMG>` hanno lo stesso significato)
- 3 tipi di tag:
 - tag di *apertura* (marcatore iniziale)
`<nome-tag>`
 - tag di *chiusura* (marcatore finale)
`</nome-tag>`
 - tag *autochiudente* (marcatore unico)
`<nome-tag>`
o anche `<nome-tag/>` (per compatibilità con XHTML)

Elementi e tag

- **Elemento** = insieme formato da tag di apertura, testo e tag di chiusura corrispondente
- Esempio di elemento B:
`<b>del testo</b>`
- A volte si usa il termine “tag” erroneamente, per indicare un elemento (composto da due tag)

Attributi dei tag (1)

- ogni occorrenza di tag (di apertura) può contenere assegnazioni di **attributi**
- **assegnazione:** `nome-attributo="valore"`
 - es. ``
- gli attributi assegnabili per ogni tag sono **finiti** (ad eccezione di `data-*`)
- le virgolette sono necessarie se si inseriscono valori multipli per lo stesso attributo (separati da spazi)
- alcuni attributi sono assegnabili con un insieme finito di valori
 - caso particolare: attributi **booleani**

Attributi dei tag (2)

- struttura del tag con attributi:

```
<nome-tag attributo1=valore1  
        attributo2="valore2 valore3"  
        attributoBooleano ...>
```

- esistono attributi **globali**
- alcuni attributi sono **obbligatori** (vanno assegnati)

Attributi globali

Tra i più usati / importanti:

- ID (non confondere con NAME)
- STYLE
- CLASS
- TITLE
 - permette di specificare una informazione associata all'elemento
 - i browser visualizzano tale informazione (pop-up) quando il puntatore del mouse passa sopra al link

Struttura di un documento HTML

```
<html>      <-- inizio del documento
  <head>
  ...
  ...      <-- intestazione del documento
</head>
<body>
  ...      <-- corpo del documento
  ...
</body>
</html>     <-- fine del documento
```

Informazione e meta-informazione

- corpo del documento = contiene **l'informazione** (il documento stesso)
- intestazione del documento = contiene **meta-informazione** (cioè informazioni **sul** documento)
 - esempi:
 - autore del documento
 - parole chiave
 - “titolo” del documento
 -

Semantica di HTML

Il “significato” di un documento HTML è dato da due componenti:

1. aspetto del documento
2. “contenuto” (rispetto ai tag) del documento

La semantica “immediata” di un documento HTML è la sua visualizzazione sul browser

- varia in base al browser (sia in input che in output)
- considerare tutti i tipi di user agent (browser testuali, screen reader, ...)
- agli scraper non interessa la resa grafica (SEO!)

Contenuto e presentazione

- Problema: distinguere tra
 - **contenuto** del documento
 - **presentazione** (o aspetto) del documento
- È molto importante poter individuare il contenuto di un documento indipendentemente dalla formattazione del documento
- Nelle intenzioni, HTML doveva mantenere separati i due aspetti. Nella realtà, non è così:
 - i marcatori sono usati anche per dare attributi di formattazione al testo
 - i vari browser “interpretano” il codice HTML in modo diverso

HTML e browser HTML

I principali web browser, specie in passato, hanno influito sull'evoluzione di HTML:

- imponendo nuovi elementi del linguaggio
- “rifiutando” (cioè non supportando) nuovi elementi del linguaggio
- Problemi principali:
 - differente interpretazione di HTML
 - presenza di vecchie versioni dei browser

Creare documenti HTML

Documento HTML = testo ASCII

(analogo ad un programma sorgente JAVA)

Modalità di creazione di un documento HTML:

- con un editor per testo ASCII (es. blocco note o Wordpad sotto Windows)
- editor avanzati o IDE
- con un “editor WYSIWYG” o “editor HTML” (es. FrontPage, Dreamweaver)
- con un editor di testi “normale” che permette di esportare i documenti in HTML (es. Word)
- Content Management System o CMS (es. Joomla, Drupal)

Editor HTML

- Permette di editare il documento vedendo direttamente come verrà visualizzato dal browser
- Facilità di utilizzo:
 - non è necessario conoscere il linguaggio HTML
- Limiti nella realizzazione:
 - non tutte le potenzialità di HTML possono essere utilizzate
- Editor HTML professionali possono essere utilizzati al massimo solo conoscendo il codice HTML

3. HTML di base

Sommario

- **intestazione**
- formattazione testo
- link
- oggetti e immagini
- tabelle
- frame

Parte intestazione (1)

- contiene una serie di informazioni necessarie al browser per una corretta interpretazione del documento, ma non visualizzate all'interno dello stesso:
- tipo di HTML supportato
- titolo della pagina
- parole chiave (per motori di ricerca)
- link base di riferimento
- stili (comandi di formattazione)

Parte intestazione (2)

Elementi principali:

- DOCTYPE (non un vero e proprio elemento)
- HTML (elemento radice)
- HEAD
- TITLE
- META

Parte intestazione (3)

```
<!DOCTYPE html>
<html>
  <head>
    <meta name="keywords" content="HTML, parte
      intestazione, meta-informazione">
    <meta name="author" content="Lorenzo Marconi">
    <meta name="generator" content="Blocco note di
      Windows 11">
    <title>Pagina web di prova</title>
  </head>
  ...
</html>
```

Parte intestazione (4)

DOCTYPE:

```
<!DOCTYPE html>
```

- fornisce l'informazione sul tipo di documento
- deve essere il primo elemento ad aprire il documento
- è obbligatorio (in HTML5)
- (nelle versioni precedenti a HTML5 doveva indicare una DTD, dichiarazione di tipo di documento)

Parte intestazione (5)

META:

```
<meta name="keywords" content="HTML, parte  
intestazione, meta-informazione">
```

```
<meta name="author" content="Lorenzo Marconi">
```

```
<meta name="generator" content="Blocco note di  
Windows 11">
```

- fornisce meta-informazioni sul contenuto del documento
- usate dai motori di ricerca per classificare il documento
- non è obbligatorio

Parte intestazione (6)

META:

- **Attributi supportati:**
 - `charset`: per specificare la codifica del documento
(es. `charset="UTF-8"`)
 - `name` : per specificare la codifica del documento
(es. `name="viewport"`)
 - `http-equiv` : per simulare l'HTTP response header
(es. `http-equiv="content-security-policy"` o
`http-equiv="content-type"`)
 - `content` : per specificare un valore associato a `name` o
`http-equiv`

Parte intestazione (7)

TITLE:

```
<title>Pagina web di prova</title>
```

- Titolo della pagina
- Compare sulla barra del titolo della finestra del browser
- Usato dai motori di ricerca
- Usato nella visualizzazione di “bookmark” (siti preferiti)

Parte intestazione (8)

Altri tag:

- `<style>`: per applicare uno stile definito localmente
- `<link>`: per specificare un collegamento con un'altra risorsa (es. favicon, foglio di stile o versione alternativa di un documento)
- `<script>`: per specificare script da eseguire lato client
- `<base>`: per specificare l'URL di base per i link nella pagina e/o le azioni di default per aprirli

HTML di base

- intestazione
- **formattazione testo**
- link
- oggetti e immagini
- tabelle
- frame

Corpo del documento

- Memorizza il contenuto del documento (la parte visualizzata all'utente del browser)

`<body>`

...

`</body>`

- gli attributi di `<body>` configurano alcuni parametri di visualizzazione del documento (in realtà il loro utilizzo è «deprecato»)

Tag contenitori

- `<div>...</div>` contenitore di tipo **block**
- `<span>...</span>` contenitore di tipo **inline**
- entrambi possono essere usati per applicare delle proprietà (es. classi CSS) ad un certo contenuto
- non supportano attributi che non siano globali

Tag logici e fisici

- tag **fisico** = ha il compito di formattare il documento
- tag **logico** = dà una struttura al documento, ed è indipendente dalla visualizzazione
- esempio: elemento `<address>`
 - permette di specificare che una parte di testo è un indirizzo
 - viene visualizzato come testo in corsivo
 - per il browser è come usare `<i>`
 - ma `<ADDRESS>` aggiunge “semantica” al documento

Tag puramente fisici

- `<B>` (bold):
 - permette di visualizzare testo in neretto
- `<I>` (italic):
 - permette di visualizzare testo in corsivo
- `<U>` (underlined):
 - permette di sottolineare il testo
- `<S>` (strike out):
 - permette di sbarrare il testo
- definiscono attributi **fisici** di formattazione

Selezione del font

- è un tag fisico
- **deprecato** da W3C: non dovrebbe mai essere usato (al suo posto vanno usate le proprietà di CSS)
- definisce il modo in cui deve essere visualizzata una parte di testo:
 - tipo di carattere
 - colore
 - grandezza

Attributi del tag

- FACE
 - determina il tipo di carattere (font) usato
 - font disponibili: courier, times, arial, verdana, ...
- SIZE
 - determina la grandezza dei caratteri
 - si esprime con numeri assoluti (da 1 a 7) o relativi
- COLOR determina il colore

esempio:

```
<FONT FACE="verdana" SIZE="+1" COLOR="green">  
testo in verde</FONT>
```

Apici e pedici

- <SUB> (subscript)
 - permette di scrivere testo come “pedici”
- <SUP> (superscript)
 - permette di scrivere “apici”
- esempio:

`I<SUB>5</SUB> = 2<SUP>4</SUP>`

viene visualizzato come

$$I_5 = 2^4$$

Header

- `<H1>,<H2>,...,<H6>`
- Permettono di inserire titoli (o intestazioni) all'interno del documento
- il testo tra `<Hn>...</Hn>` viene evidenziato dal browser
- 6 livelli di titoli
- 6 differenti livelli di enfaticizzazione del testo
- Non confondere con `<head>` e `<header>`!

Testo preformattato

- Tag <PRE>
- permette di visualizzare il testo esattamente come è scritto (“preformattato”) nel file sorgente HTML
 - viene usato un font a larghezza fissa ("fixed-width")
 - vengono preservati tutti gli spazi e le andate a capo

Testo preformattato con <PRE>

Codice HTML:

```
<PRE>
begin
  if
  then
end;
      I<SUB>5</SUB>      =  2<SUP>4</SUP>
</PRE>
```

Visualizzazione:

```
begin
  if
  then
end;
      I5      =  24
```

Tag logici (1)

- **<ADDRESS>**
 - marcatura usata per indirizzi (mail, email, telefono,...)
- **<Q>** e **<BLOCKQUOTE>**
 - usati per inserire nel testo citazioni da un altro testo o autore (rispettivamente brevi o lunghe)
- **<CITE>**
 - usato per la fonte della citazione
- **** e ****
 - “enfaticizzano” il testo all’interno del tag
- **<VAR>**, **<KBD>**, **<CODE>** e **<SAMP>**
 - utilizzati per variabili, testo inserito da tastiera, righe di codice di programmazione e output di un programma

Tag logici (2)

- **<ABBR>**
 - usato per abbreviazioni e acronimi
- **** e **<INS>**
 - usati per indicare parti di testo cancellate o inserite nel documento (verranno indicate sbarrate e sottolineate, risp.)
- **<DFN>**
 - usato per dare una definizione

Separare e allineare il testo

- `<P>` (paragraph)
 - crea un paragrafo all'interno del testo
- `<BR>` (break)
 - interruzione di riga (“a capo”)
- `<HR>` (horizontal row)
 - aggiunge una riga orizzontale al testo
 - usato per separare parti di testo

Liste puntate

- `<UL>` (unordered list)
 - produce un elenco (lista) di elementi (parti di testo)
 - ogni elemento è evidenziato all'inizio da un simbolo grafico (di solito cerchietto o quadratino)
- `<LI>` (list item)
 - per identificare un elemento della lista

- **esempio:**

```
<UL>  
  <LI>Primo elemento</LI>  
  <LI>Secondo elemento</LI>  
  <LI>Terzo elemento</LI>  
</UL>
```

Liste numerate

- `<OL>` (ordered list)
 - produce un elenco (lista) di elementi (parti di testo)
 - ogni elemento è evidenziato all'inizio dal numero d'ordine all'interno della lista
- `<LI>` (list item) (come per liste puntate)
- esempio:

```
<OL>  
  <LI>Primo elemento</LI>  
  <LI>Secondo elemento</LI>  
  <LI>Terzo elemento</LI>  
</OL>
```

Liste numerate

Oltre al numero progressivo, si possono usare altre indicizzazioni per le liste puntate

Uso dell'attributo TYPE di :

- `<OL TYPE="A">` indicizza con lettere alfabetiche maiuscole
- `<OL TYPE="a">` indicizza con lettere alfabetiche minuscole
- `<OL TYPE="I">` indicizza con numeri romani maiuscoli
- `<OL TYPE="i">` indicizza con numeri romani minuscoli

Fino ad HTML4 era possibile farle qualcosa di simile anche con (ad es. specificando `type="square"`), ora questa funzionalità è demandata a CSS

Liste annidate

Esempio di liste annidate:

```
<ol>
  <li>gruppo di nomi:
    <ul>
      <li>nome a</li>
      <li>nome b</li>
    </ul>
  </li>
  <li>gruppo di nomi:
    <ul>
      <li>nome c</li>
      <li>nome d</li>
      <li>nome e</li>
    </ul>
  </li>
</ol>
```

Reso come:

1. gruppo di nomi
 - nome a
 - nome b
2. gruppo di nomi:
 - nome c
 - nome d
 - nome e

HTML di base

- intestazione
- formattazione testo
- **link**
- oggetti e immagini
- tabelle
- frame

Link ipertestuali

Tag <A> (anchor)

- permette l'inserimento di link ipertestuali all'interno del documento
- non confondere con <link>!
- attributo principale: HREF ("hypertext reference")
 - specifica la URL che viene associata al link
- il testo tra <A> e viene associato a tale URL
 - cliccando su tale testo il browser accede alla URL
- Esempio:

```
<A HREF="http://www.virgilio.it">Visita  
virgilio.it</A>
```

Anchor

...

```
<p id="par1">un certo paragrafo del documento</p>
```

...

```
<a href="#par1">vai all'anchor "par1" nel  
documento</a>
```

...

Con l'anchor par1 si può anche accedere direttamente dall'esterno del documento:

```
<a href="www.diag.uniroma1.it/index.html#par1">
```

```
vai al paragrafo "par1" del documento index.html  
del sito diag.uniroma1.it </a>
```

Attributo target di <a>

Attributo TARGET di <A>:

- permette di specificare dove deve essere visualizzata la URL associata al link

valori principali di TARGET:

- `_blank`: visualizza la URL collegata in una nuova finestra (o tab) del browser
- `_top`: visualizza nella stessa finestra, eliminando tutti i frame presenti (vedere sezione sui frame)
- `_parent`: visualizza nel frame genitore

HTML di base

- intestazione
- formattazione testo
- link
- **oggetti e immagini**
- tabelle
- frame

Immagini

- HTML permette di inserire immagini nel documento
- Tag (singolo)
- permette di inserire nel documento una immagine, memorizzata in un file (o URL) a parte
- i browser riconoscono i principali formati immagine (es. JPG, GIF, PNG)

Attributi del tag

- SRC
 - specifica il nome della URL contenente l'immagine
 - es. `<IMG SRC="foto1.jpg">`
- WIDTH larghezza (in pixel o percentuale)
- HEIGHT altezza (in pixel o percentuale)
 - se WIDTH e HEIGHT mancano, l'immagine viene visualizzata nelle sue dimensioni originali
- ALT
 - permette di aggiungere un commento testuale associato all'immagine

L'attributo ALT

- Permette di aggiungere una informazione testuale all'immagine
- Il testo viene visualizzato dai browser (pop-up)
- Necessario per i browser solo testuali
- Oppure per la navigazione con immagini disabilite

es. `<IMG SRC="topolino.gif" ALT="disegno che raffigura Topolino e Pippo">`

Mappe cliccabili

- Una immagine può essere associata ad un link
 - es. `<a href="pippo.htm"></a>`
- Si possono associare due o più link alla stessa immagine
- **Mappe cliccabili**: associare zone diverse dell'immagine a diversi link
- 2 tipi:
 - **lato server**, es. ISMAP (sconsigliato)
`<a href="server.php"></a>`
 - **lato client**, es. USEMAP

Mappe cliccabili raster

```
<IMG SRC="img1.gif" alt="Una bella mappa"  
  usemap="#immagine1">
```

```
<map name="immagine1">  
  <area shape="rect" coords="0,0,250,150"  
    href="page1.html" alt="Page 1">  
  <area shape="circle" coords="300,150,50"  
    href="page2.html" alt="Page 2">  
  <area shape="poly"  
    coords="350,50,450,150,400,200,300,100"  
    href="page3.html" alt="Page 3">  
  <area shape="default" href="#">  
</map>
```

Generare mappe cliccabili

- Tipi di aree definibili con `usemap`:
 - `rect`
 - `circle`
 - `poly`
- difficili da definire a mano
- uso di programmi (editor di mappe)
 - es. MAPedit

Mappe cliccabili SVG

- Un'alternativa più flessibile al tag <MAP> è usare <SVG>
- **SVG** è un formato di immagine **vettoriale** basato su XML e indipendente da HTML
 - alcuni tag standard: <circle>, <rect>, <polygon>, <linearGradient>, <path>
- Il tag <SVG> (introdotto in HTML5) permette di usare vari tag SVG per comporre immagini vettoriali
- Nota: Per le mappe geografiche esistono vari metodi per creare mappe complesse e interattive (es. Leaflet.js, OpenLayers, Google Maps API) ma richiedono tutti l'uso di JavaScript (quindi esulano dall'argomento di queste slides)

Mappe cliccabili SVG

```
<svg viewBox="0 0 500 300">
  <a href="pagina1.html">
    <rect x="10" y="10" width="75" height="50"
      class="area" />
  </a>
  <a href="pagina2.html">
    <circle cx="150" cy="40" r="30"
class="area" />
  </a>
  <a href="pagina3.html">
    <polygon points="100,100 175,125 75,125"
      class="area" />
  </a>
</svg>
```

Oggetti

- Tag `<object>` e `<embed>`
 - Permettono l'inclusione di oggetti (risorse) generici nel documento HTML
 - Esempio:

```
<embed type="text/html"
      src="page1.html" width="600"
      height="400">
```

Entità

- Le **entità** nascono per ovviare al problema di rappresentare simboli non-ASCII e simboli riservati
- molte entità hanno un nome simbolico associato
- l'invocazione di un'entità inizia con “&” e termina con “;”
 - esempio: `©` per rappresentare ©
- le entità si possono anche rappresentare mediante i loro codici Unicode (in decimale o esadecimale (es.: `{` `�`))
- il simbolo “&” si rappresenta con `&`

Entità

- storicamente, per chi scriveva in italiano era importante ricordare le entità per le lettere accentate

- `à` à
- `è` è
- `é` é
- `ì` ì
- `ò` ò
- `ù` ù

Il simbolo “<”

- < è un simbolo particolarmente speciale in HTML (apertura dei tag)
- < si rappresenta con `<`;
- > si rappresenta con `>`;

HTML di base

- intestazione
- formattazione testo
- link
- oggetti e immagini
- **tabelle**
- frame

Tabelle

- Rappresentano informazione in forma tabellare (righe e colonne) nei documenti HTML
- Molto utilizzate anche come strumento di formattazione di testi e/o immagini
- HTML permette una gestione piuttosto potente delle tabelle

Elementi relativi alle tabelle

- TABLE
 - definisce la tabella
- TR
 - suddivide i campi in righe all'interno della tabella
- TD
 - definisce un campo “dati” all'interno della tabella
- TH
 - definisce campi intestazione all'interno della tabella
- THEAD, TBODY, TFOOT

L'elemento `<table>`

Racchiude tutta l'informazione relativa ad una **tabella**

Tag da usare dentro `<table>`:

- `<tr>` per le righe
- `<td>` e `<th>` per le celle (usare dentro `<tr>`)
- `<caption>` per la didascalia
- `<thead>`, `<tbody>` e `<tfoot>` per suddividere il contenuto della tabella
- `<colgroup>` per raggruppare delle colonne

L'elemento <TD>

<TD> permette la definizione di un singolo campo (cella)

Attributi (opzionali):

- `colspan` (num. colonne occupate dalla cella)
- `rowspan` (num. righe occupate dalla cella)
- `headers` (header a cui si riferisce la cella)

L'elemento <TR>

- Divide le celle in righe
- Esempio:

```
<TABLE>
```

```
<TR>
```

```
<TD>cella 1</TD>
```

```
<TD>cella 2</TD>
```

```
<TD>cella 3</TD></TR>
```

```
<TR>
```

```
<TD>cella 4</TD>
```

```
<TD>cella 5</TD>
```

```
<TD>cella 6</TD></TR>
```

```
</TABLE>
```

cella 1	cella 2	cella 3
cella 4	cella 5	cella 6

Esempio

```
<TABLE>
<TR>
  <TD colspan="2">A</TD>
  <TD>B</TD>
</TR>
<TR>
  <TD>C</TD>
  <TD rowspan="2">D</TD>
  <TD>E</TD>
</TR>
<TR>
  <TD>F</TD>
  <TD>G</TD>
</TR></TABLE>
```

A		B
C	D	E
F		G

L'elemento <TH>

- Come <TD> ma va usato per specificare campi **intestazione**
- permette di dare più “semantica” alla tabella
- da un punto di vista di presentazione, per i browser non c'è differenza
- stessi attributi di <TD>

Esempio

```
<TABLE>
<TR>
  <TH>nome</TH>
  <TH>cognome</TH>
  <TH>età</TH></TR>
<TR>
  <TD>Mario</TD>
  <TD>Rossi</TD>
  <TD>36</TD></TR>
<TR>
  <TD>Paola</TD>
  <TD>Bianchi</TD>
  <TD>32</TD></TR>
</TABLE>
```

nome	cognome	età
Mario	Rossi	36
Paola	Bianchi	32

L'elemento <CAPTION>

- Permette di associare una didascalia alla tabella
- esempio:

```
<TABLE>
<CAPTION>Elenco degli impiegati:</CAPTION>
<TR>
  <TH>nome</TH>
  <TH>cognome</TH>
  <TH>età</TH></TR>
<TR>
  <TD>Mario</TD>
  <TD>Rossi</TD>
  <TD>36</TD></TR>
<TR>
  <TD>Paola</TD>
  <TD>Bianchi</TD>
  <TD>32</TD></TR>
</TABLE>
```

Elenco degli impiegati:

nome	cognome	età
Mario	Rossi	36
Paola	Bianchi	32

Elementi di raggruppamento righe

- `<THEAD>`, `<TBODY>`, `<TFOOT>`
- Permettono di suddividere l'informazione contenuta in una tabella per righe
- Permettono la gestione separata di intestazione e contenuto
- Permettono di raggruppare il contenuto della tabella in più gruppi (più occorrenze di `<TBODY>...</TBODY>`)
- Struttura non visualizzata dai browser (occorrono fogli di stile)

Esempio

```
<TABLE>
<THEAD>
  <TR><TH>nome</TH>
    <TH>cognome</TH>
    <TH>et&agrave;</TH></TR>
</THEAD>
<TBODY>
  <TR>
    <TD>Mario</TD>
    <TD>Rossi</TD>
    <TD>36</TD></TR>
  <TR>
    <TD>Paola</TD>
    <TD>Bianchi</TD>
    <TD>32</TD></TR>
</TBODY>
</TABLE>
```

Raggruppamento delle colonne

- `<COLGROUP>`, `<COL>`
- Permettono di suddividere l'informazione contenuta in una tabella per colonne
- Struttura non visualizzata dai browser (occorrono fogli di stile)

HTML di base

- intestazione
- formattazione testo
- link
- oggetti e immagini
- tabelle
- **frame**

Frame

- Frame = riquadro (zona rettangolare) della finestra di visualizzazione del browser
- ogni frame è gestito in modo indipendente dal browser (come una finestra a sé stante)
- Nelle versioni precedenti di HTML erano presenti diversi elementi per organizzare la finestra in frame
- Nel seguito vedremo l'unico tag tuttora supportato, ovvero `<IFRAME>`

L'elemento <iframe>

- Il tag <IFRAME> («inline frame») permette di definire un frame (riquadro) in qualsiasi punto di un documento HTML
- struttura:

```
<IFRAME src="http://www.uniroma1.it"  
width="500" height="300">
```

codice HTML alternativo (per i
browser che non supportano iframe)
detto di *fallback*

```
</IFRAME>
```

4. Form

Form (modulo)

- usati per inviare informazioni via WWW
- il modulo viene compilato dall'utente (sul browser)
- quindi viene inviato al server
- un programma apposito sul server elabora il modulo
- in genere tale programma invia una “risposta” all'utente (pagina web, email, ecc.)

L'elemento `<form>`

Apre e chiude il modulo e raccoglie il contenuto dello stesso

Esempio:

```
<FORM method="post"
  action="http://www.diag.uniroma1.it/cgi-
  bin/nome_script.cgi">
```

- **attributo ACTION:** specifica la URL della risorsa (programma) che elabora la form

L'attributo method

`method="get"`

- i dati inseriti nella form vengono spediti al server e separati in due variabili
- le coppie nome-campo-form=valore-inserito compaiono alla fine della URL usata per l'invio del messaggio (i dati sono quindi visibili già nella URL)

- Esempio (Google):

```
https://www.google.com/search?q=linguaggi+e+tecnologie+per+il+web&ie=utf-8&oe=utf-8&client=firefox-b
```

- per questo metodo il numero massimo di caratteri contenuti nella form è circa 3000

L'attributo method

```
method="post"
```

- i dati vengono ricevuti direttamente dal programma (lato server) senza un preventivo processo di decodifica
- questa caratteristica fa sì che lo script possa leggere una quantità illimitata di caratteri

Tag da usare in un modulo

Vengono definiti tramite gli elementi:

- `INPUT` (campi editabili, checkbox, radio, ecc.)
- `TEXTAREA` (area di testo)
- `SELECT` (creazione di menu di opzioni)
 - da usare in combinazione con `OPTION` (e eventualmente `OPTGROUP`)
- `BUTTON` (simula un tasto da premere)
- `LABEL` (etichetta associata a un altro elemento)
- `OUTPUT` (risultato di una certa computazione)
- `FIELDSET` (insieme di campi)
 - da usare in combinazione con `LEGEND`

L'elemento INPUT

- Permette l'inserimento di campi editabili e/o modificabili nel modulo
- Attributi principali:
 - TYPE: determina il tipo di campo
 - NAME: chiave da passare al server
 - VALUE: valore da passare al server
 - MINLENGTH / MAXLENGTH
 - SIZE: larghezza specificata in numero di caratteri

Attributo TYPE di <INPUT>

- Attributo TYPE: determina il tipo di campo
- Principali possibili valori di tale attributo:
 - HIDDEN
 - TEXT
 - PASSWORD
 - CHECKBOX
 - RADIO
 - SUBMIT
 - RESET
 - IMAGE

TYPE="HIDDEN"

- Usato per consegnare al server delle informazioni non visibili dall'utente (se non nel sorgente della pagina)

- Ad esempio

```
<INPUT type="hidden" name="user" value="pippo">
```

invia al server la coppia `user=pippo` (non visibile dall'utente)

TYPE="TEXT"

```
<INPUT type="text" name="nome"  
  maxlength="40" size="33"  
  value="inserisci nome">
```

- crea i tipici campi di testo
- usato soprattutto per informazioni non predefinite che variano di volta in volta
- attributi di <INPUT> nel caso TYPE=TEXT:
 - MAXLENGTH (num. max caratteri inseribili)
 - SIZE (larghezza campo all'interno della pagina)
 - VALUE (valore di default che compare nel campo)

TYPE="PASSWORD"

```
<INPUT type="password" name="pwd"  
    maxlength="40" size="33">
```

- crea i campi di tipo password
- vengono visualizzati asterischi al posto dei caratteri
- il valore *non* viene criptato (problema per la sicurezza)

TYPE="CHECKBOX"

```
<INPUT type="checkbox" name="eta"  
size="3" VALUE="YES" CHECKED>
```

- crea campi booleani (si/no)
- crea delle piccole caselle quadrate da spuntare o da lasciare in bianco
- VALUE impostato su YES significa che di default la casella è spuntata
- CHECKED controlla lo stato iniziale della casella, all'atto del caricamento della pagina

TYPE="RADIO"

```
<INPUT type="radio" name="giudizio"  
value="sufficiente">
```

```
<INPUT type="radio" name="giudizio"  
value="buono">
```

```
<INPUT type="radio" name="giudizio"  
value="ottimo">
```

- usato per selezionare una tra alcune scelte
- tutte le scelte con lo stesso “name” (altrimenti una scelta non esclude le altre)

TYPE="SUBMIT"

```
<INPUT type="submit" value="spedisci">
```

- crea un bottone che invia il modulo al server
- la lunghezza del bottone dipende dalla lunghezza del valore di `VALUE`

TYPE="RESET"

```
<INPUT type="reset" value="reimposta">
```

- crea un bottone che resetta i campi del modulo
- i dati inseriti vengono eliminati
- la lunghezza del bottone dipende dalla lunghezza del valore di `VALUE`

TYPE="IMAGE"

```
<INPUT type="image" SRC="bottone.gif">
```

- come SUBMIT, ma crea un bottone tramite l'immagine (URL) specificata in SRC

TYPE="FILE"

- Usato per caricare un file
- Va specificato `enctype="multipart/form-data"` nel tag form
 - per default, il valore di `enctype` è `application/x-www-form-urlencoded`

Attributi associati:

- ACCEPT
 - serve filtrare i file per estensione o media type
- MULTIPLE (da HTML5)
 - attributo booleano per abilitare caricamenti multipli

L'elemento TEXTAREA

```
<TEXTAREA cols="40" rows="5" name="commento">  
    Testo iniziale  
</textarea>
```

Attributi associati:

- MAXLENGTH
- COLS
- ROWS

L'elemento SELECT

```
<SELECT size="2" cols="4" name="giudizio">  
  <OPTION selected value="nessuna">  
  <OPTION value="buono"> Buono  
  <OPTION value="sufficiente"> Sufficiente  
  <OPTION value="ottimo"> Ottimo  
</select>
```

- Permette la creazione di menu con scelte multiple
- L'attributo `size` permette di indicare quante opzioni devono essere visibili contemporaneamente
 - `size=1` genera un menù a tendina

L'elemento FIELDSET

```
<FIELDSET name="giudizio">  
  <LEGEND>Dati dello studente:</legend>  
  <input type="text" name="matricola">  
  <input type="text" name="anno-iscrizione">  
  <input type="text" name="numero-esami">  
</fieldset>
```

- permette la creazione di un gruppo di campi all'interno della form
- l'elemento `<legend>` permette di inserire una descrizione (o titolo) per il gruppo di campi

L'elemento **BUTTON**

- Permette l'inserimento di un **pulsante cliccabile** (non necessariamente di tipo "submit")
- A differenza del tag `<INPUT>`, tramite il tag `<BUTTON>` va aperto e chiuso, e può contenere del codice HTML (che apparirà all'interno del pulsante)
- L'attributo `TYPE` può essere valorizzato con "button", "submit" or "reset" (per default è "submit")

L'elemento LABEL

- Permette l'inserimento di un'**etichetta**, che può essere associata ad un elemento di input tramite il suo ID
- Esempio:

```
<LABEL for="myInput">Spuntami!</label>  
<input type="checkbox" id="myInput">
```
- Cliccando sul testo "Spuntami!" verrà selezionata la checkbox associata

Esempio di form

cognome e nome:	
data di nascita:	
CAP:	
codice fiscale:	
studente lavoratore:	☐
foto:	Sfoggia... Nessun file selezionato.
giudizio:	☐ sufficiente ☐ buono ☒ ottimo
descrizione lavoro:	
Invia form	Reset form

Codice HTML della form

```
<form action="" method="post" name="eseform"
  enctype="multipart/form-data">
<table>
<tr>
  <td>cognome e nome:</td>
  <td><input type="text" name="nome" size="50" maxlength="50">
</td></tr>
<tr>
  <td>data di nascita:</td>
  <td><input type="date" name="ddn"></td></tr>
<tr>
  <td>CAP:</td>
  <td><input type="number" name="cap" size="5" maxlength="5"></td></tr>
<tr>
  <td>codice fiscale:</td>
  <td><input type="text" name="cf" size="16" maxlength="16"></td></tr>
<tr>
  <td>studente lavoratore:</td>
  <td><input type="checkbox" name="lavoratore"></td></tr>
```

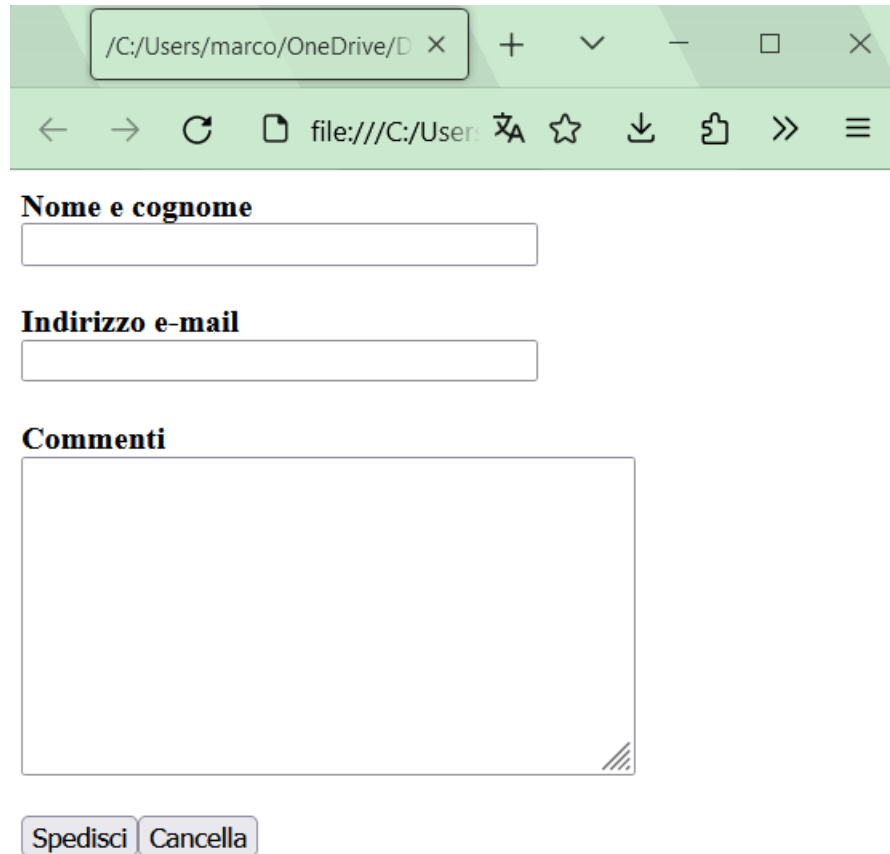
Codice HTML della form (continua)

```
<tr>
  <td>foto:</td>
  <td><input type="file" name="foto"></td></tr>
<tr>
  <td>giudizio:</td>
  <td>
    <input type="radio" name="giudizio" value="sufficiente">sufficiente
    <input type="radio" name="giudizio" value="buono">buono
    <input type="radio" name="giudizio" value="ottimo">ottimo
  </td></tr>
<tr>
  <td>descrizione lavoro:</td>
  <td><textarea name="descr" cols="60" rows="10"></textarea></td></tr>
<tr>
  <td><input type="submit" value="Invia form"></td>
  <td><input type="reset" value="Reset form"></td></tr>
</table>
</form>
```

Esempio 2

```
<form
  action="http://www.coder.com/code/mailform/mailform.pl.cgi"
  method="post">
  <input type="hidden" name="mailform_id" value="val_7743">
  <input type="hidden" name="mailform_subject" value="il mio
primo form">
  <b>Nome e cognome</b><br>
  <input type="text" name="mailform_name" size=33><br><br>
  <b>Indirizzo e-mail</b><br>
  <input type="text" name="mailform_from" size=33><br><br>
  <b>Commenti</b><br>
  <textarea name="mailform_text" rows="10"
cols="42"></textarea><br><br>
  <input type="submit" value="spedisci"><input type="reset"
    value="cancella">
</form>
```

Esempio 2 (cont.)



The image shows a web browser window with a light green header. The address bar contains the path `/C:/Users/marco/OneDrive/D` and a close button. Below the address bar is a navigation bar with icons for back, forward, refresh, and a file icon, followed by the text `file:///C:/User`. The main content area contains a form with three sections: **Nome e cognome** with a single-line text input, **Indirizzo e-mail** with a single-line text input, and **Commenti** with a large multi-line text area. At the bottom of the form are two buttons: **Spedisci** and **Cancella**.

Commenti

`<!--questo è un commento HTML-->`

- I commenti permettono di inserire del testo nascosto all'interno del codice sorgente.
- I commenti NON sono elementi HTML

HTML5: breve storia

- **HTML5** è il successore di HTML 4 e XHTML 1.0, standardizzati dal W3C nel 1999 e 2000
- Nel 2002 il W3C decide che la futura versione 2.0 di XHTML debba rimpiazzare HTML (questo implica la **non-retrocompatibilità** del nuovo linguaggio con HTML 4), ed essere *document-oriented* e non *application-oriented*
- Nel 2004 si forma il consorzio **WHATWG** in opposizione al W3C e all'adozione di XHTML
 - Slogan di WHATWG: “Don’t break the Web”

HTML5: breve storia (segue)

- WHATWG rilascia nel 2008 il primo draft del suo HTML5 (rilasciato ufficialmente dal 2014)
- Nel frattempo il W3C torna sui suoi passi, e finisce per sposare la visione WHATWG. Il progetto XHTML 2.0 viene chiuso, e parte il lavoro per la standardizzazione di HTML5
- Nel 2012 WHATWG rilascia l'“HTML5 living standard”, ovvero una specifica di HTML5 che verrà lasciata evolvere continuamente (senza rilascio di versioni specifiche)

HTML5: breve storia (segue)

- Nel 2014 il W3C rilascia il suo standard HTML5
- Nel 2016 il W3C rilascia HTML 5.1
- Nel 2017 il W3C rilascia HTML 5.2
- Nel 2019 W3C e WHATWG concordano che in futuro l'unico riferimento comune per i due consorzi sarà il “living standard” di WHATWG

HTML5 vs. HTML 4

- Nuove marcature **strutturali** e «semantiche»
- Nuovi tag e attributi per le **form**
- Tag specifici per incorporare **audio** e **video**
- Regioni "disegnabili" (**canvas**)
- Altri nuovi tag
- Supporto ai **microdati**
- Nuove **API** (comprese nello standard)

Nuove API

- HTML5 aumenta significativamente le API messe a disposizione degli script
- Gli obiettivi sono:
 - In generale, accrescere le possibilità offerte alla programmazione lato client
 - Standardizzare alcuni tipi più comuni di operazioni effettuate lato client
 - Aggiornare le API alle necessità dei media agent più recenti (smartphone, tablet)

Cache e usi offline

- permettono di salvare copie locali di un insieme di risorse, allo scopo di permettere al browser di eseguire applicazioni anche in modalità offline
- vecchio approccio: elencare in un file manifest tutte le risorse da scaricare e mettere nella cache
- ora lo standard consiglia l'uso dei **Service Worker**

Script async/defer

```
<script src="..."></script>
```



```
<script src="..." async></script>
```



```
<script src="..." defer></script>
```



Cookie

- I **cookie** (in particolare **cookie HTTP**) sono una tecnica nata per permettere ad un server di (re)identificare un client
 - vengono inviati ad ogni connessione HTTP
 - espediente per mantenere una sessione attiva
- I **cookie di terze parti** sono un uso controverso dei cookie. vengono usati principalmente per:
 - Pubblicità mirata
 - Retargeting
 - Analisi e statistiche (es. Google Analytics)
 - Funzionalità social

Web Storage + IndexedDB API

- Oggetti **localStorage** e **sessionStorage** (accessibili come array associativi)
 - Estendono le capacità di memorizzazione dei cookie
 - Possono memorizzare solo stringhe (testo): per memorizzare oggetti arbitrari occorre serializzarli (ad esempio tramite JSON)
 - I dati *non* vengono inviati automaticamente al server
- **IndexedDB** consente di gestire database lato-client
 - database → Object Store
 - object-oriented (*non* relazionale)

WebSocket API

- Permettono di instaurare una connessione dati **bidirezionale** tra web client e web server
- L'oggetto `WebSocket` serve a gestire una connessione TCP **full-duplex** con un server
 - Il metodo `send(x)` **invia** il testo `x` al server
 - La funzione associata all'event handler `onmessage` viene eseguita quando dal server **arriva** un messaggio
- Utili ogni qual volta si ha bisogno di reagire in **tempo reale** ad eventi remoti (es. chat, notifiche push, giochi online, ecc.)
- **Bassa latenza**: utili anche come sostituto di download multipli, poiché si evita il continuo handshake delle connessioni HTTP
- Il principale svantaggio è la **bassa scalabilità**: ogni client mantiene una connessione persistente, quindi server con molti utenti devono avere risorse adeguate

EventSource API

- Permettono la creazione dei **Server-Sent Events (SSE)**
- Comunicazione **server** → **client** in tempo reale
- Alternativa ai WebSocket, quando non è necessario instaurare una comunicazione bidirezionale
- Com'è possibile, se HTTP è *connectionless*?
 - Caso particolare di richiesta HTTP (il canale rimane aperto in download):

```
GET /events HTTP/1.1
Host: esempio.com
Accept: text/event-stream
Cache-Control: no-cache
Connection: keep-alive
```

Geolocation API

- Servono a gestire dati geospaziali, tipicamente la posizione (anche se questa è effettivamente disponibile solo su alcuni user agent)
- La proprietà `geolocation` dell'oggetto `navigator` ha due metodi per ottenere la posizione corrente:
 - `getCurrentPosition`
 - `watchPosition`
 - (il secondo metodo differisce dal primo perché restituisce una nuova posizione ogni volta che questa cambia)
 - Attenzione! Funzionano solo in ambienti sicuri (quindi con HTTPS o localhost)

Audio/Video e nuove API

- HTML5 permette la gestione nativa (ovvero senza ricorrere a plug-in del browser) di contenuti multimediali
- tag `<video>`: permette di inserire un contenuto video
- tag `<audio>`: permette di inserire un contenuto audio
- il formato dei video e degli audio supportati dipende dai browser, tuttavia esistono dei formati di riferimento (mp4, webm, ogg per i video)
- esistono delle nuove API per video e audio che permettono la gestione di questo tipo di risorse da script

Canvas

- elemento `<canvas>` per dichiarare una zona della pagina su cui è possibile disegnare, tramite nuove API
- si può disegnare una canvas in un contesto 2D o in un contesto 3D
- le API per disegnare (in contesto 2D) si dividono in
 - metodi path (linee, archi, ...)
 - metodi modificatori (rotazioni, traslazioni, ...)
 - metodo `drawImage` (inserisce un'immagine)
 - metodi per scrivere testo
 - metodi di colorazione: riempimento, ombreggiatura, gradienti, ...

Nuove API (cont'd)

- Media API
- Notification API
- Drag & Drop API
- ... e molte altre!

DOCTYPE

In HTML5 il DOCTYPE non fa più riferimento a una DTD, e viene semplificato nel modo seguente:

```
<!DOCTYPE html>
```

Nuovi attributi globali

- `contenteditable` (abilita modifica del contenuto testuale)
- `data-*` (attributi definibili da utente)
- `enterkeyhint` (testo per il tasto "invio" nella tastiera virtuale dei dispositivi mobili)
- `inert` (disabilita l'interazione con l'elemento)
- `draggable` (attenzione è, un "falso" booleano!)
- `hidden` (nasconde l'elemento)
- `spellcheck`
- **e altri...**

Tag strutturali e semantici

- `<header>`
- `<footer>`
- `<section>`
- `<article>`
- `<nav>`
- `<aside>`
- `<hgroup>`
- `<details>` e `<summary>`
- `<time>`
- `<meter>` e `<progress>`
- `<picture>`



Esempio d'uso dei tag di HTML5 per marcare semanticamente la struttura della pagina

Altri tag

- `<figure>`
- `<figcaption>`
- `<wbr>`
- `<menu>` (alternativa a `<ul>`)
- `<mark>`
- `<output>`

Form

- Nuovi attributi ed input type per le form sono stati introdotti in HTML5
- L'obiettivo principale è quello di permettere di definire le più comuni forme di validazione di form lato client direttamente nel documento HTML (cioè senza bisogno di aggiungere codice JavaScript)
- Sono stati inoltre aggiunti tipi di elementi utili soprattutto nei nuovi media agent (smartphone e tablet)

Autofocus, placeholder, form

Nuovi attributi:

- **autofocus** (booleano): all'apertura della form, il focus va sull'elemento che ha dichiarato autofocus
- **placeholder** (per elementi input o textarea): valore che all'apertura della form compare sul campo editabile (N.B.: non corrisponde ad un valore (value) iniziale del campo, viene solo visualizzato all'inizio)
- **form**: attributo che permette di associare un elemento ad una form (specificandone l'ID). È così possibile, ad esempio, dichiarare un campo che appartiene a più form, o dichiarare un campo *all'esterno* di un elemento form

Required, autocomplete

- **required** (booleano): rende obbligatoria la compilazione dell'elemento al momento del submit della form a cui l'elemento appartiene
- **autocomplete**: attributo che può assumere due valori:
 - **on** = il browser può effettuare l'**autocompletion** di questo campo, cioè completare il campo in maniera automatica usando i valori precedentemente inseriti in questo campo
 - **off** = il browser non può fare l'autocompletion
 - se autocomplete non viene assegnato, viene usato il *default* del browser (di solito è on)

Multiple, pattern

- **multiple** (booleano): permette al campo di accettare valori multipli

- esempio:

```
<form action="demo_form.asp">  
  Indirizzo email: <input type="email" multiple>  
  Fotografie: <input type="file" multiple>  
  <input type="submit">  
</form>
```

- **pattern**: viene assegnato ad una espressione regolare; i valori del campo devono appartenere al linguaggio denotato dall'espressione regolare

Min, max, step

- `min`: valore minimo ammesso
- `max`: valore massimo ammesso
- `step`: intervallo tra un valore ammesso e il successivo
- `novalidate` (booleano): se dichiarato sull'elemento form, indica che **non** verrà effettuata la validazione degli elementi di quella form

Nuovi input type

I seguenti input types permettono di gestire informazioni testuali di tipo specifico:

- `tel`
- `search`
- `url`
- `email`
- `color`: permette la selezione di un colore
- `number`: permette l'inserimento di un numero
- `range`: permette l'inserimento di un numero attraverso uno slider (cursore orizzontale)

Nuovi input type (segue)

Per gestire le date sono stati introdotti i seguenti input types:

- `datetime-local`
- `date`
- `month`
- `week`
- `time`

Il tag `<datalist>`

- Permette di definire un campo testuale sul quale il browser può effettuare **autocompletion** usando un insieme predefinito di valori (l'utente però è libero di scrivere un valore al di fuori dell'elenco predefinito)
- esempio:

```
<input type="text" name="giudiz" list="listagiudizi">  
  
<datalist id="listagiudizi">  
  <option value="insufficiente">  
  <option value="sufficiente">  
  <option value="discreto">  
  <option value="buono">  
  <option value="ottimo">  
</datalist>
```

Supporto ai microdati

- i microdati servono a creare un tagging «semantico» di porzioni del documento
- sono basati vocabolari che definiscono identificatori di proprietà relative ad un (micro-)dominio di interesse
- si utilizzano gli attributi HTML `itemscope`, `itemtype`, `itemprop` e `itemid`

Esempio: il vocabolario Person

Estratto del vocabolario <https://schema.org/Person>:

Property	Description
givenName	Given name. In the U.S., the first name of a Person.
familyName	Family name. In the U.S., the last name of a Person.
image	An image of the item. This can be a URL or a fully described ImageObject. [ereditato da Thing]
jobTitle	The job title of the person (for example, Financial Manager).
height	The height of the item.
weight	The weight of the product or person.
parent	A parent of this person. Supersedes parents.
affiliation	An organization that this person is affiliated with. For example, a school/university, a club, or a team.
address	Physical address of the item.
birthDate	Date of birth.
birthPlace	The place where the person was born.

Microdati in HTML5: esempio

...

```
<div itemscope itemtype="https://schema.org/Person">
```

Benvenuti nella home page di

```
<span itemprop="givenName">Lorenzo</span>
```

```
<span itemprop="familyName">Marconi</span>,
```

```
<span itemprop="jobTitle">docente</span> alla
```

```
<span itemprop="affiliation">Sapienza Università di  
Roma</span>.
```

```
Indirizzo di casa: <address itemprop="address">221B Baker  
Street, Londra</address>
```

```
</div>
```

...

Microdati in HTML5: elementi annidati

...

```
<div itemscope itemtype="https://schema.org/Person">
```

```
  Benvenuti nella home page di
```

```
  <span itemprop="givenName">Lorenzo</span>
```

```
  <span itemprop="familyName">Marconi</span>,</div>
```

```
  <span itemprop="jobTitle">docente</span> alla
```

```
  <span itemprop="affiliation">Sapienza Università di  
Roma</span>.
```

```
  Indirizzo di casa: <address itemprop="address" itemscope  
itemtype="https://schema.org/PostalAddress">
```

```
    <span itemprop="streetAddress">221B Baker Street</span>,</div>
```

```
    <span itemprop="addressLocality">Londra</span>
```

```
  </address>
```

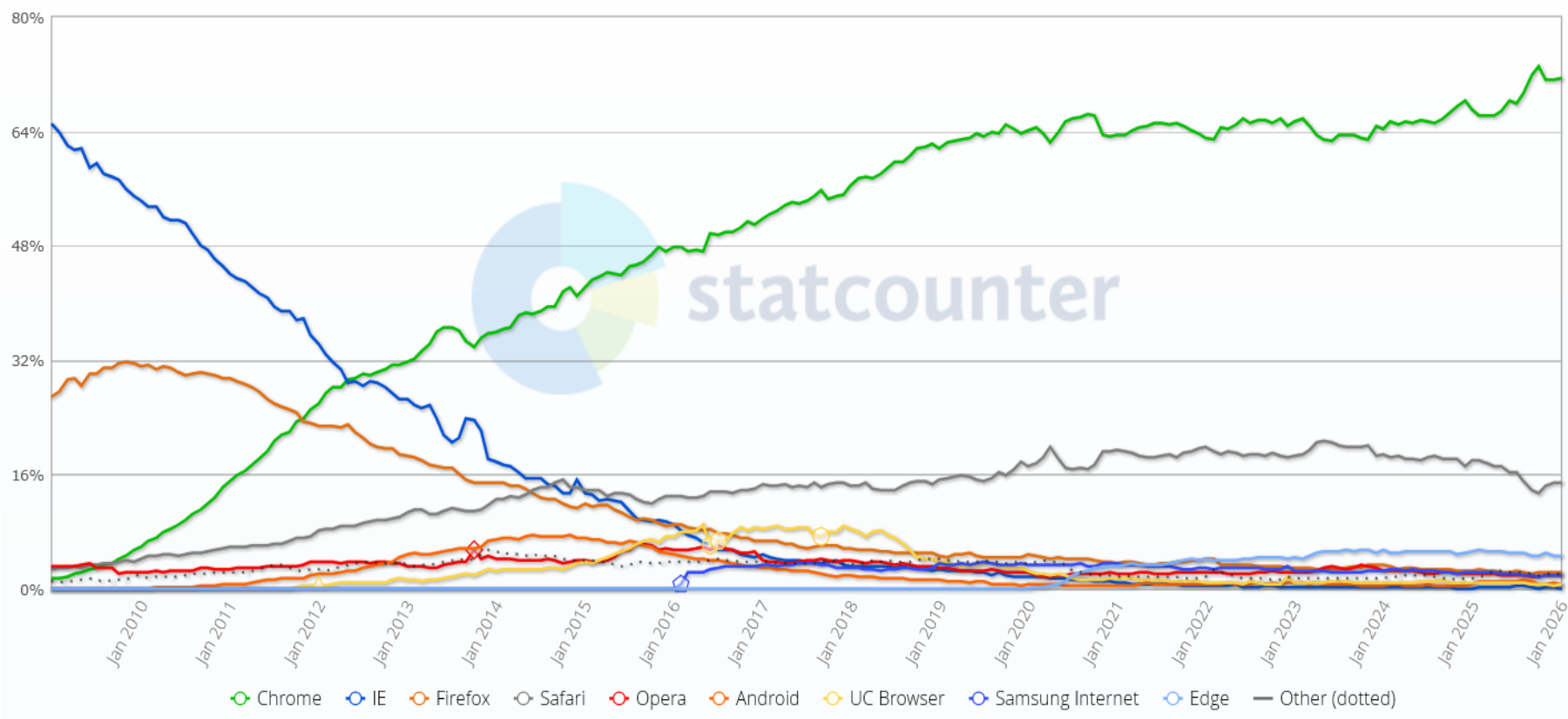
```
</div>
```

...

Diffusione dei web browser, 2009-2026

Browser Market Share Worldwide

Jan 2009 - Jan 2026

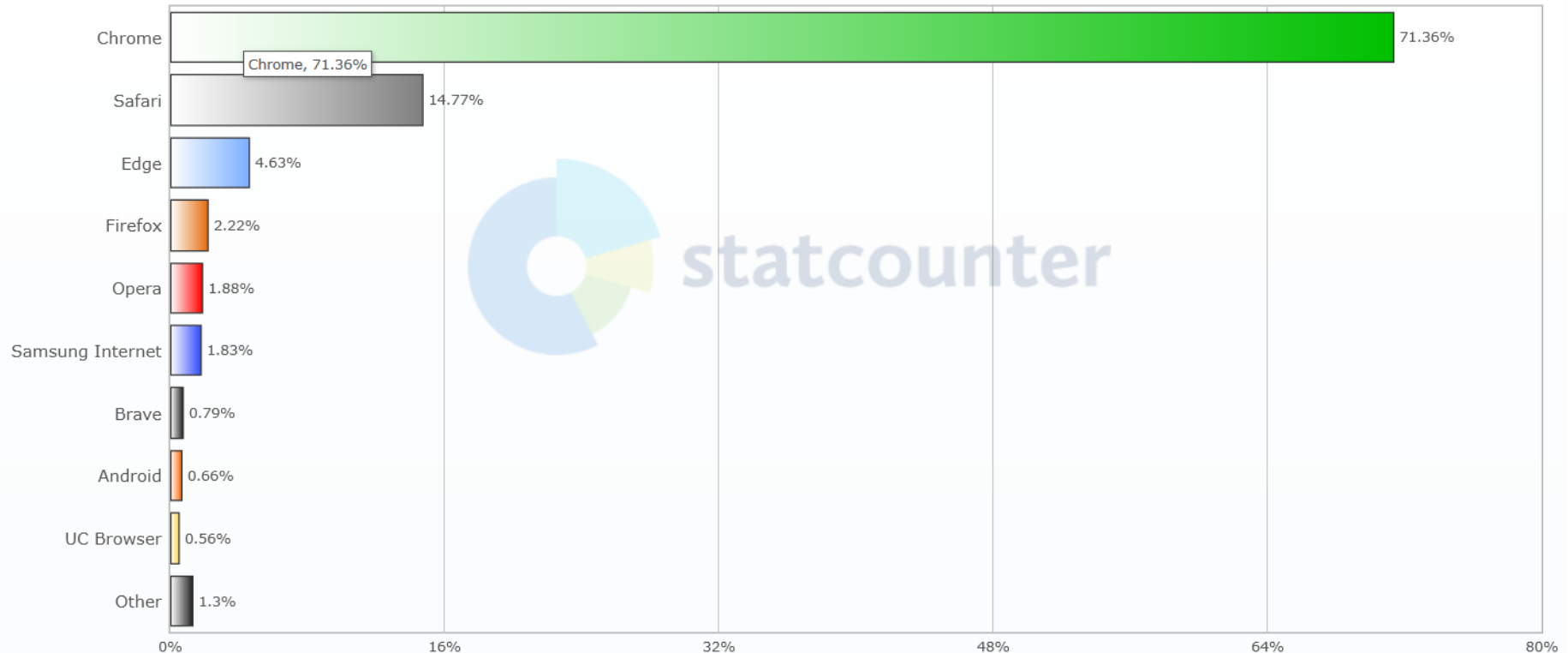


(fonte: gs.statcounter.com)

I web browser più diffusi (gennaio 2026)

Browser Market Share Worldwide

Jan 2026



(fonte: gs.statcounter.com)

Riferimenti

- Raccomandazione ufficiale W3C su HTML:
 - www.w3.org/TR/html5/
(attualmente è equivalente al link successivo)
- HTML living standard (WHATWG):
 - html.spec.whatwg.org/multipage/
- Guida di MDN sulle Web API:
 - [developer.mozilla.org/docs/Learn/JavaScript/Client-side web APIs](http://developer.mozilla.org/docs/Learn/JavaScript/Client-side_web_APIs)
 - developer.mozilla.org/docs/Web/API

Riferimenti

- Guida online per HTML e altre tecnologie Web:
 - www.w3schools.com
- Sito italiano per HTML e altre tecnologie Web:
 - www.html.it
- Microdati, data vocabulary:
 - www.schema.org